

RAMAIAH INSTITUTE OF TECHNOLOGY

BENGALURU – 560054

(Affiliated to, Visvesvaraya Technological University, Belagavi)



Mini Project report on

**Crack Detection Drone Using ESP32-CAM & ML-Based
Vision System**

Submitted in partial fulfillment of the requirement for the award of

BACHELOR OF ENGINEERING
In
MECHANICAL ENGINEERING

Submitted by

PRATEEK GHALE	-	1MS23ME086
SAMARTH SHARMA	-	1MS23ME098
SHASHANKA C	-	1MS23ME106
UTKARSH GOEL	-	1MS23ME123

Under the guidance of

DR. ARUNA KUMARA P.C.
Assistant Professor
Department of Mechanical Engineering,
Ramaiah Institute of Technology, Bengaluru.

**Department of Mechanical Engineering,
Ramaiah Institute of Technology, Bengaluru – 560054
May 2026**



RAMAIAH
Institute of Technology

Department of Mechanical Engineering

CERTIFICATE

Certified that the mini project work entitled “**Crack Detection Drone Using ESP32-CAM & ML-Based Vision System**” carried out by **PRATEEK GHALE** (1MS23ME086), **SAMARTH SHARMA** (1MS23ME098), **SHASHANKA C** (1MS23ME106), **UTKARSH GOEL** (1MS23ME123), The bonafide students of Ramaiah Institute of Technology Bengaluru, in fulfillment for the award of Bachelor of Engineering in Mechanical Engineering of the Visvesvaraya Technological University Belagavi, during the academic year 2025-2026. It is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in the Report deposited in the departmental library.

The project report has been approved as it satisfies the academic requirements with respect to the Mini Project Work prescribed for the said Degree.

Signature of the guide
DR. ARUNA KUMARA
P.C.
Assistant Professor

Signature of the HOD
Dr. K.R.V. SUBRAMANIAN

Signature of the Principal
Dr. N.V.R. NAIDU

Name of External Examiners:

Signature and Date:

1. _____

2. _____



RAMAIAH
Institute of Technology

Department of Mechanical Engineering

DECLARATION

We hereby declare that the entire work embodied in this project has been independently carried out by us under the supervision of **DR. ARUNA KUMARA P.C. Assistant Professor**, Department of Mechanical Engineering, **Ramaiah Institute of Technology**, Bengaluru, in partial fulfillment of the requirement of the Bachelor's Degree in **Mechanical Engineering**. We further declare that the project report has not been submitted either in part or in full to any other university for the award of any Degree.

By,

PRATEEK GHALE
(1MS23ME086)

SAMARTH SHARMA
(1MS23ME098)

SHASHANKA C
(1MS23ME106)

UTKARSH GOEL
(1MS23ME123)



Department of Mechanical Engineering

ACKNOWLEDGEMENT

First and foremost, we would like to express my deep gratitude for the invaluable guidance of my Guide Dr. Aruna Kumara P.C., Assistant Professor, Department of Mechanical Engineering, Ramaiah Institute of Technology, Bangalore, for all his guidance, help and ongoing support throughout the course of this work.

We immensely grateful to Dr. K.R.V. Subramanian, Professor and Head, Department of Mechanical Engineering, Ramaiah Institute of Technology, Bangalore, without whose inspiration and invaluable support my research would never have taken off.

We extremely thankful to Dr. N.V.R. Naidu, Principal, Ramaiah Institute of Technology, for creating an atmosphere of Research and Development in the College and encouraging us to initiate into the research activities.

Prateek Ghale

Samarth Sharma

Shashanka C

Utkarsh Goel

ABSTRACT

Structural integrity monitoring of buildings, bridges, and road surfaces is a critical yet labor-intensive process. This project presents an autonomous crack-detection drone integrating an ESP32-CAM module with an OV2640 image sensor mounted on a custom drone frame. A trained machine learning model processes real-time imagery to identify surface cracks with high accuracy.

The system is designed to reduce manual inspection effort by enabling rapid aerial screening of hard-to-reach surfaces and capturing visual evidence in real time. Its compact embedded architecture supports onboard image acquisition and inference, making it suitable for lightweight inspection missions where speed, accessibility, and low deployment cost are important.

The project's key innovations include an integrated low-cost sensing platform, edge-based vision processing, and a drone payload optimized for inspection in constrained environments. By combining hardware design, embedded vision, and ML-based classification, the solution aims to improve inspection efficiency while maintaining portability and practical deployment readiness.

TABLE OF CONTENTS

SI No.	Description	Page No.
	CERTIFICATE	i
	DECLARATION	ii
	ACKNOWLEDGEMENT	iii
	ABSTRACT	iv
1	INTRODUCTION	1
	1.1 ACCIDENTS DUE TO CRACKS	1
2.	2.1 LITERATUE SURVEY	4
	2.1.1 EARLY UAV IMPLEMENTATIONS & PHOTOGRAMMETRY	4
	2.1.2 ALGORITHMIC OPTIMIZATION (CNNs)	5
	2.1.3 HARDWARE SHIFT (LOW-COST EDGE COMPUTING)	6
	2.1.4 FLIGHT STABILITY & AERODYNAMICS	6
	2.2 PROBLEM DEFINITION	8
	2.3 OBJECTIVES	9
3.	METHODOLOGY	11
	3.1 BLOCK DIAGRAM	11
	3.2 HARDWARE USED	12
	3.2.1 MICROCONTROLLER	12

3.2.2 CAMERA SENSOR	13
3.2.3 APPLICATION OF CAMERA MODULE	14
3.2.3.1 ROCK-CLIMBING ROBOTS WITH BIO GRIPPERS	14
3.2.3.2 UNMANNED AERIAL VEHICLE (UAV)	15
3.2.3.3 CAD DESIGN	16
3.2.3.4 CFD ANALYSIS	17
3.3 SOFTWARE REQUIREMENTS	21
3.3.1 EMBEDDED FIRMWARE & DEVELOPMENT ENVIRONMENT	21
3.3.2 DESIGN AND SIMULATION SOFTWARE	22
3.3.3 GROUND STATION MACHINE LEARNING ENVIRONMENT	22
3.3.4 OPERATING SYSTEM PLATFORMS	23
3.3.5 DETAILED SOFTWARE & ENVIRONMENT SPECIFICATIONS	24
4. EXPERIMENTAL DETAILS AND PROCEDURE	26
4.1 WORKING PRINCIPLE	26
4.2 SOFTWARE IMPLEMENTATION	27
4.2.1 ARDUINO FIRMWARE	27
4.2.1.1 CODE EXECUTION BREAKDOWN	27
4.2.2 GROUND STATION SERVER	29

4.2.3 THE CANVAS BRIDGE	30
4.2.4 SOFTWARE ML MODEL	31
5. RESULT & DISCUSSIONS	33
5.1 EXPERIMENTAL RESULTS	33
5.2 DISCUSSIONS	34
5.2.1 ADVANTAGES	34
5.2.2 DISADVANTAGES	34
5.2.3 APPLICATIONS	35
6. CONCLUSIONS	37
7. SCOPE OF FUTURE WORK	38
REFERENCES	40

LIST OF FIGURES

SI No.	Description	Page No.
FIG. 1.1	MANUAL WORKFORCE REPAIRING CRACKS	1
FIG. 1.2	PARADIGM SHIFT IN STRUCTURAL INSPECTION	3
FIG. 3.1	BLOCK DIAGRAM OF THE SYSTEM	11
FIG. 3.2.1	ESP 32 CAM MODULE	12
FIG. 3.2.2	FT232RL PROGRAMMING BOARD	12
FIG. 3.2.3.1	ROCK-CLIMBING ROBOTS WITH BIO GRIPPERS	14
FIG. 3.2.3.2	DRONE WITH PRIMUS X5	15
FIG. 3.2.3.3	CAD MODEL OF DRONE	16
FIG. 3.2.3.4.1	AIRFLOW DISTRIBUTION OVER DRONE PAYLOAD (VELOCITY[M/S] ALONG Z DIRECTION)	17
FIG. 3.2.3.4.2	AIRFLOW DISTRIBUTION OVER DRONE PAYLOAD (PRESSURE[PA] ALONG Z DIRECTION)	18
FIG. 3.2.3.4.3	TEMPERATURE (FLUID)[K] ON CANOPY	18
FIG. 3.2.3.4.4	TEMPERATURE (FLUID)[K] ON PROPELLER BLADES	19
FIG. 3.2.3.4.5	PRESSURE [PA] ON CANOPY	19

FIG. 3.2.3.4.6 PRESSURE [PA] ON PROPELLER BLADE	20
FIG. 3.2.3.4.7 SURFACE PARAMETERS	20
FIG. 3.2.3.4.8 PRESSURE [PA] ANALYSIS	21
FIG. 4.1 HTML5 BASED CRACK DETECTION CANVAS	26
FIG. 4.2.1.1 CODE EXECUTION FLOWCHART	28
FIG. 4.2.2.1 HOSTING LOCAL SERVER ON TERMINAL	29
FIG. 4.2.2.2 EXECUTION INSTRUCTIONS	30
FIG. 4.2.4 MACHINE LEARNING INFERENCE WORKFLOW	32
FIG. 5.1 MODEL ACCURACY OVER EPOCHS	34

LIST OF TABLES

SI No.	Description	Page No.
TABLE 1.1	ECONOMIC AND OPERATIONAL IMPACT OF STRUCTURAL FAILURES	2
TABLE 2.1	SUMMARY OF EVALUATED LITERATURE	7
TABLE 3.1.1.1	ESP32-CAM MICROCONTROLLER SPECIFICATIONS	12
TABLE 3.2.1.2	ESP32-CAM PIN CONFIGURATION	13
TABLE 3.2.2	OV2640 CAMERA MODULE SPECIFICATIONS	14
TABLE 3.2.3.2	SPECIFICATIONS OF DRONE	16
TABLE 3.2.3.4	SURFACE PARAMETERS OF FIG. 3.2.3.4.6	21
TABLE 5.1	EXPERIMENTAL PERFORMANCE METRICS	33
TABLE 5.2.2	POWER CONSUMPTION ANALYSIS OF PAYLOAD	35

CHAPTER 1

INTRODUCTION

1.1 Accidents Due to Crack



Fig. 1.1 Manual Workforce Repairing Cracks

The deterioration of civil infrastructure presents a formidable challenge, both economically and in terms of public safety. Structural anomalies, most notably surface cracking, are primary indicators of underlying distress in concrete structures, bridges, pavement, and architectural facades. If these micro-fissures propagate unchecked, they result in catastrophic structural yield. Historically, deferred maintenance and undetected structural faults have culminated in disastrous collapses, engendering severe loss of life and massive financial liabilities.

Traditional methodologies for detecting these anomalies depend predominantly on manual visual inspection. This necessitates deploying engineers on scaffolding, heavy machinery, or ropes to traverse large vertical structures. Such methods are not only perilous but also fraught with human subjectivity and fatigue. Furthermore, the immense cost of halting traffic or shutting down facilities during inspection periods magnifies the economic loss.

To mitigate these systemic inefficiencies and safety risks, the civil engineering industry is experiencing a paradigm shift toward automated, non-destructive testing (NDT) methodologies. The advent of Unmanned Aerial Vehicles (UAVs) equipped with high-resolution optical sensors and edge-computing modules has revolutionized structural health monitoring. Unlike manual inspections, drone-based systems can dynamically access challenging angles and elevated facades without requiring massive infrastructural scaffolding or endangering human lives. By coupling these aerial platforms with lightweight Machine Learning algorithms, such as Convolutional Neural Networks (CNNs), structural defects can be autonomously detected, quantified, and geolocated in real time, transforming reactive maintenance into a proactive, data-driven science.

Table 1.1 Economic and Operational Impact of Structural Failures

Impact Category	Description	Consequence
Human Safety	Collapse of bridges and residential buildings due to missed cracks.	Fatalities and severe injuries
Direct Economic Impact	Cost of emergency structural reconstruction and legal liabilities.	Millions of dollars in unplanned expenditure
Indirect Economic Impact	Supply chain disruption, traffic gridlocks, and business downtime.	Loss of local commerce and productivity
Operational Risk	Injuries sustained by inspectors performing manual high-altitude checks.	Increased insurance and worker compensation costs

To mitigate these systemic inefficiencies, there is a distinct transition toward automated Unmanned Aerial Vehicle (UAV) systems. A UAV can dynamically access challenging angles without risking human life or requiring massive infrastructural scaffolding.

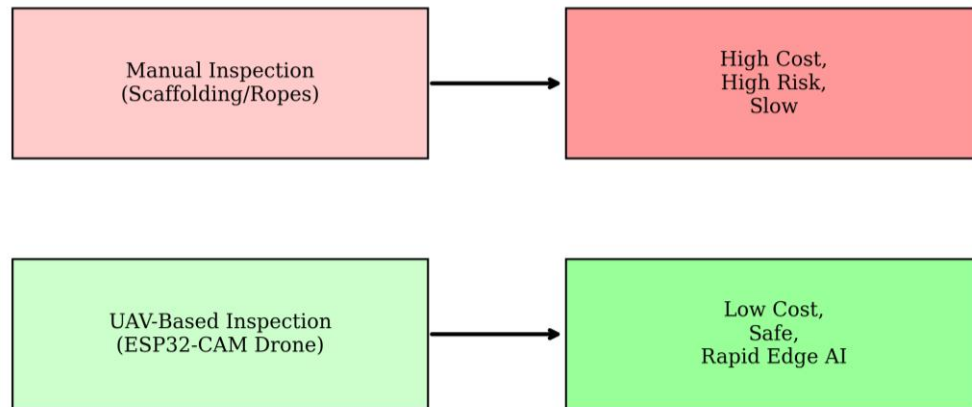


Fig. 1.2 Paradigm Shift in Structural Inspection

CHAPTER 2

LITERATURE SURVEY AND PROBLEM DEFINITION

2.1 Literature Survey

2.1.1 Early UAV Implementations & Photogrammetry

The landscape of automated structural monitoring has expanded rapidly with the proliferation of drones and deep learning. Early implementations focused on drones as mere mobile camera platforms. Kim et al. [1] demonstrated the fundamental viability of UAV-driven structural crack detection and localized determination, laying the groundwork for aerial photogrammetry in civil engineering. Following this trajectory, Zhang et al. [2] and Li et al. [3] expanded these concepts to deep-learning-based UAV systems for building and pavement inspections, highlighting the necessity of robust data collection paradigms.

- **Relevance:** This initial phase was critical in shifting the industry away from dangerous, manual rope-access inspections toward safer, scalable, and remote aerial surveys.
- **Findings:** These early studies collectively demonstrated that commercial off-the-shelf (COTS) drones, coupled with foundational deep learning architectures, could successfully map macro-level structural faults and drastically reduce inspection timelines.
- **Limitations:** However, these systems were primarily constrained by their reliance on ideal lighting conditions, high-resolution (and heavy) camera payloads, and a crippling dependency on massive, manually annotated datasets. Furthermore, their reliance on GPS for localization rendered them largely ineffective in GPS-denied environments, such as indoor industrial facilities.
- **Future Work:** Future research in this foundational area must prioritize multi-modal sensor fusion (e.g., integrating lightweight LiDAR with RGB cameras) and the development of unsupervised or semi-supervised learning techniques to eliminate the manual annotation bottleneck.

2.1.2 Algorithmic Optimization (CNNs)

Progressing into algorithmic optimization, researchers began deploying Convolutional Neural Networks (CNNs). Wang et al. [6] introduced novel approaches utilizing Deep CNNs for UAV imagery, while Zhao et al. [7] developed 'DenxiDeepCrack', a trainable network specifically optimized for automatic road crack detection. Further innovations include the YOLOv8-CBAM model proposed by Chen et al. [8], emphasizing attention mechanisms for enhanced building crack detection, and the use of VGG16 models by Wei et al. [9] for concrete pavements. Qu et al. [10] addressed data scarcity by establishing transfer learning methods tailored for infrastructure cracks. Additionally, Vinayagam et al. [4] and Ahmed et al. [5] thoroughly documented crack detection and broader image-based methodologies for structural surface inspection via drones.

- **Relevance:** This algorithmic evolution directly addresses the core computational challenge of automated monitoring: accurately isolating complex, hairline crack morphologies from highly textured and noisy background materials like concrete and brick.
- **Findings:** The implementation of attention mechanisms (such as CBAM) and transfer learning dramatically increased mean Average Precision (mAP) and reduced false positives, proving that highly accurate models could be trained even with limited infrastructure-specific datasets.
- **Limitations:** Despite these gains, models like VGG16 and deep YOLO variants suffer from a high parameter count, requiring heavy cloud-based or terrestrial GPU processing. This computational overhead creates latency and makes real-time, on-device inference difficult for standalone drone operations.
- **Future Work:** Consequently, future algorithmic optimization must focus on model pruning, quantization, and the design of ultra-lightweight neural architectures (e.g., MobileNet variants) specifically tailored to run efficiently on resource-constrained drone hardware.

2.1.3 Hardware Shift (Low-Cost Edge Computing)

Simultaneously, the hardware paradigm shifted toward low-cost, edge-computing nodes. Pawar et al. [11], Rahat et al. [12], and Ramesh et al. [13] pioneered the development of affordable mini quadcopters and surveillance drones centered entirely around the ESP32 and ESP32-CAM architecture. This minimized cost while maintaining essential IoT capabilities. Ali et al. [14] further validated real-time edge inference specifically on the ESP32 for structural health, while Martinez et al. [15] provided crucial power consumption analyses for these remote sensing nodes.

- **Relevance:** This shift is vital for democratizing structural health monitoring, moving away from expensive proprietary drone fleets toward ultra-low-cost, decentralized, and scalable IoT drone swarms.
- **Findings:** These investigations confirmed that microcontrollers like the ESP32-CAM can successfully perform basic image acquisition and execute lightweight edge inference (e.g., binary crack/no-crack classification) while adhering to strict, battery-limited power budgets.
- **Limitations:** Nevertheless, this hardware paradigm is severely restricted by the inherent limitations of the ESP32, namely low RAM, minimal processing throughput, and low frame-rate image capture. These constraints currently preclude the native execution of the complex, multi-class CNNs discussed in earlier literature.
- **Future Work:** To bridge this gap, future hardware development requires the integration of dedicated, low-power Neural Processing Units (NPUs) alongside edge microcontrollers, as well as the exploration of onboard energy-harvesting mechanisms to extend operational flight times during continuous monitoring tasks.

2.1.4 Flight Stability & Aerodynamics

Flight stability and aerodynamic integrity are also critical when mounting external payloads. Kim et al. [16] conducted foundational aerodynamic design and CFD analyses of quadcopter camera mounts to reduce turbulence. Control stability, essential for clear image capture, was heavily investigated by Smith & Patel [17], Liu et al. [18], Kumar & Singh [19], and Patel et

al. [20], focusing on PID algorithms, active disturbance rejection, MPU6050 integration with ESP32, and optimized ESC interfaces.

- **Relevance:** Aerodynamic and control stability are the literal foundations of reliable data collection; the most advanced deep learning algorithms are rendered useless if the airborne payload captures motion-blurred or highly distorted imagery.
- **Findings:** Researchers revealed that optimized PID tuning, coupled with active disturbance rejection and aerodynamically rigid payload mounts, can significantly dampen high-frequency rotor vibrations and ensure the optical stability required for high-fidelity photogrammetry.
- **Limitations:** However, traditional PID controllers often struggle to adapt dynamically to sudden, unpredictable wind gusts or the complex, turbulent ground-effect environments frequently encountered near large vertical structures (e.g., building facades, bridge pillars).

Future Work: Moving forward, flight control research must pivot toward reinforcement-learning-based adaptive flight controllers that can anticipate aerodynamic disturbances, as well as the development of morphing drone frames capable of physically stabilizing against chaotic wind shear during close-proximity inspections.

Table 2.1 Summary of Evaluated Literature

Author & Year	Key Focus Area	Primary Hardware/Algorithm
Kim et al., 2021 [1]	Crack Location Determination	General UAV / Photogrammetry
Zhang et al., 2025 [2]	Building Crack Detection	UAV / Deep Learning
Wang et al., 2022 [6]	UAV Image Crack Detection	Deep CNN

Chen et al., 2024 [8]	Enhanced Detection Accuracy	YOLOv8-CBAM
Pawar et al., 2026 [11]	Affordable Mini Quadcopter	ESP32 Architecture
Ali et al., 2023 [14]	Edge Inference on Concrete	ESP32 / Edge AI
Kim et al., 2022 [16]	Aerodynamics & CFD	Camera Mounts / Quadcopter
Kumar & Singh, 2020 [19]	Flight Stabilization	ESP32 & MPU6050

2.2 Problem definition

Despite the advancements documented in literature, a critical gap remains. High-end UAVs utilizing advanced CNNs (like YOLOv8) demand substantial onboard computational power or continuous high-bandwidth cloud connectivity, making them prohibitively expensive for widespread, routine deployment in developing regions. Conversely, ultra-low-cost ESP32 drones often lack the integrated machine learning frameworks required to perform autonomous defect classification. The core problem is synthesizing structural aerodynamic stability, ultra-low-cost hardware (ESP32-CAM), and a decentralized, browser-based edge ML pipeline into a single, cohesive, portable unit that operates effectively on a localized network without requiring expensive cloud computing.

Compounding this technological gap are the severe occupational hazards and systemic inefficiencies intrinsic to traditional manual inspection. When structural health is assessed manually, human engineers are forced into precarious, high-altitude environments via scaffolding, suspended platforms, or industrial rope access. This directly exposes personnel to fatal fall risks, sudden aerodynamic wind-shear around tall structures, and the unpredictable danger of localized structural collapse during assessment. Furthermore, close-proximity visual inspections often subject workers to respiratory hazards from airborne concrete dust, asbestos, or toxic mold found in aging infrastructure. Beyond the physical dangers, these manual methods suffer from profound operational drawbacks. They are heavily reliant on subjective human judgment, highly prone to fatigue-induced oversight, and lack the precise, repeatable digital traceability required to accurately map micro-crack propagation over time.

The massive logistical burden of mobilizing heavy access equipment like snooper trucks coupled with the crippling indirect costs of prolonged traffic lane closures and facility downtime creates an urgent imperative for an automated, remote-sensing alternative that completely eliminates human physical risk while standardizing data collection.

2.3 Objectives

Based on the identified constraints of ultra-low-cost hardware and the computational demands of deep learning, this project aims to fulfill the following objectives:

1. **Hardware Integration:** To engineer a cost-effective multirotor chassis capable of carrying a vision payload with optimized aerodynamic stability for structural inspection.
2. **Payload CAD Design:** To develop a specialized 3D Computer-Aided Design (CAD) model of the drone's payload cradle that securely houses the ESP32-CAM while incorporating anti-vibration geometry to prevent image distortion.
3. **Computational Fluid Dynamics (CFD):** To perform comprehensive aerodynamic simulations evaluating airflow velocity and pressure distribution profiles across the drone chassis and external payload.
4. **Aerodynamic Optimization:** To analyze the CFD simulation results to iteratively refine the payload's aerodynamic footprint, minimizing parasitic drag and turbulent wakes to guarantee precise hover stability during image capture.
5. **Vision Acquisition:** To integrate the ESP32-CAM module and OV2640 sensor specifically as a dedicated, lightweight wireless video acquisition node to capture video stream.
6. **Telemetry Pipeline:** To establish a low-latency, high-bandwidth local Wi-Fi network capable of reliably broadcasting an MJPEG video stream from the drone to a ground station
7. **Browser-Based Edge Inference:** To offload machine learning computations from the microcontroller by developing a local ground-station web application capable of receiving the video stream.

8. **Model Development:** To train a custom Convolutional Neural Network (CNN) optimized for identifying irregular structural cracks across various concrete surfaces.
9. **Cross-Origin Resolution:** To implement necessary security bypasses and local hosting configurations (such as disabling strict CORS policies) to allow modern web browsers to seamlessly interface with raw IoT data streams.
10. **Canvas-Bridge Engineering:** To develop a dynamic HTML5 Canvas bridge that continuously captures frames from the incoming MJPEG stream, formats them, and feeds them into the AI engine without triggering DOM clone errors.
11. **Real-Time Overlays:** To utilize WebGL and local GPU acceleration to perform inference and render dynamic bounding boxes and confidence scores over the live video feed in real-time.
12. **Latency Reduction:** To achieve zero cloud-latency during inference by ensuring the CNN model payload is downloaded locally via the inference.js SDK before inspection begins.
13. **Scalable Architecture:** To formulate a highly scalable inspection system where multiple low-cost sensor drones can be deployed simultaneously, leveraging the robust processing power of ground-station computers rather than relying on expensive, proprietary onboard AI chips.

CHAPTER 3

METHODOLOGY- SOFTWARES USED TO MAKE

3.1 Block Diagram

The methodology entails a symbiotic integration of hardware mechanics, firmware, and machine learning software. The architecture relies on decentralized processing where the flight controller manages kinematic stability while the ESP32-CAM operates exclusively as the wireless video streamer. As illustrated in the block diagram below, this setup details the flow of power, signals, and data across the system, highlighting the specific roles of each module.

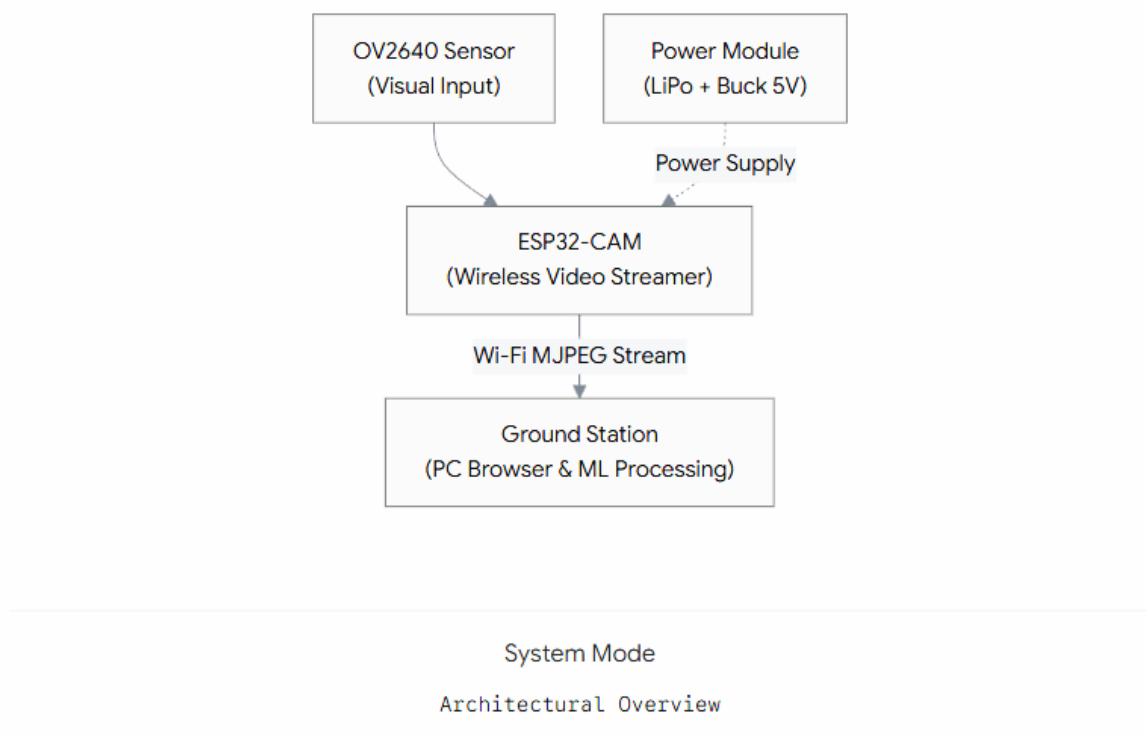


Fig. 3.1 Block diagram of the system

3.2 Hardware Used

3.2.1 Microcontroller



Figure 3.2.1 ESP 32 CAM Module

The central cognitive unit is the ESP32-CAM. Chosen for its unparalleled cost-to-performance ratio in edge AI applications, it features a dual-core 32-bit CPU and integrates directly with camera sensors programmed by FT232RL programming board.

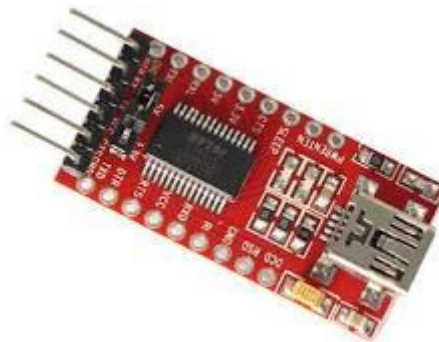


Fig. 3.2.2 FT232RL Programming Board

Table 3.2.1.1 ESP32-CAM Microcontroller Specifications

Parameter	Specification
Processor Core	Tensilica LX6 dual-core 32-bit
Clock Frequency	Up to 240 MHz
Internal SRAM	520 KB

External PSRAM	4 MB (Critical for ML image buffering)
Wireless Connectivity	802.11 b/g/n Wi-Fi, Bluetooth 4.2 / BLE
Storage	MicroSD Card Slot (up to 4GB)
Operating Voltage	3.3V (powered via 5V input pin)

Correct pin mapping is vital for programming and integration with external telemetry modules.

Table 3.2.1.2 ESP32-CAM Pin Configuration

Pin Function	ESP32-CAM Pin	Description
Power Supply	5V, GND	Main power from voltage regulator
UART Transmit	U0T (GPIO 1)	Serial TX for firmware flashing/debugging
UART Receive	U0R (GPIO 3)	Serial RX for firmware flashing/debugging
Flash Mode	GPIO 0	Pulled to GND during firmware upload
Onboard Flash LED	GPIO 4	High-power white LED for dark environments
Indicator LED	GPIO 33	Built-in red LED for status signaling

3.2.2 Camera Sensor

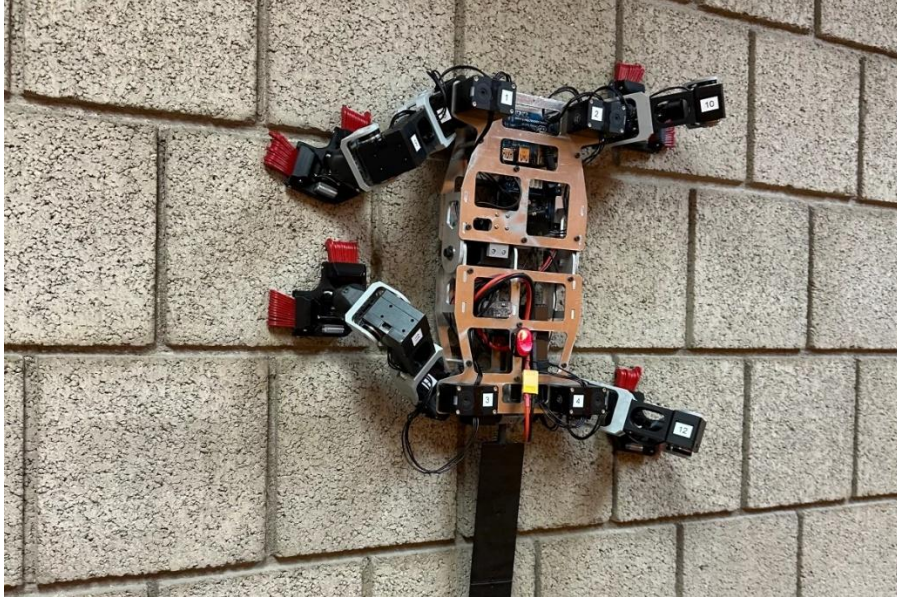
The OV2640 sensor acts as the primary optical interface. It is attached to the ESP32 via a flexible flat cable and utilizes the SCCB (Serial Camera Control Bus) protocol for configuration.

Table 3.2.2 OV2640 Camera Module Specifications

Feature	Specification
Active Array Size	1622 x 1200 (2 Megapixels)
Lens Size	1/4 inch
Output Formats	YUV422, RGB565, JPEG compressed
Maximum Frame Rate	UXGA/SXGA @ 15 fps, SVGA @ 30 fps
Sensitivity	0.6 V/Lux-sec
Power Consumption	125 mW (Active), 600 μ W (Standby)

3.2.3 Applications of Camera Module

3.2.3.1 Rock-climbing Robots with Bio Grippers

**Fig. 3.2.3.1 Rock-climbing Robots with Bio Grippers**

Beyond aerial deployment, the ESP32-CAM payload can be integrated into specialized vertical-climbing chassis, specifically those utilizing biomimetic grippers. These robotic platforms employ micro-spine arrays or gecko-inspired directional adhesives to scale non-

magnetic, highly textured surfaces—such as concrete dams, masonry bridge pillars, and natural rock faces—where traditional magnetic crawlers fail and UAVs are hindered by localized wind turbulence. In this configuration, the ultra-lightweight profile of the ESP32-CAM is highly advantageous, as climbing robots must strictly minimize payload mass to maintain shear-force adhesion against gravity. By mounting the vision module on such a crawler, engineers can conduct close-proximity, continuous edge-based visual inspections of structural fissures over extended durations, leveraging the lower continuous power consumption of crawling mechanics compared to aerial hovering

3.2.3.2 Unmanned Aerial Vehicle (UAV)



Fig. 3.2.3.2 Drone with Primus X5

While the ESP32-CAM module is highly versatile and can be mounted on a wide array of platforms including pipeline inspection crawlers, magnetic wall-climbing robots, and fixed cable-cam systems for the scope of this project, we considered its application specifically integrated into a custom multirotor Unmanned Aerial Vehicle (UAV).

A high-discharge Lithium Polymer (LiPo) battery supplies power to the high-draw Brushless DC (BLDC) motors via Electronic Speed Controllers (ESCs) along with flight controller microcontroller.

Table 3.2.3.2 Specifications of drone

Feature	Specification
Size	16 x 16 cm

Speed	3m/s
Weight	60 gms
Max Range	60 m
Flight time	8+ minutes
Rated Capacity	600mAH
Discharge Rating	25 C
Chemistry	Lithium polymer

3.2.3.3 CAD Design

Computer-Aided Design (CAD) is essential for developing the structural mounts of the UAV. The payload cradle housing the ESP32-CAM was modeled to situate the sensor centrally beneath the chassis, optimizing the center of gravity. Anti-vibration rubber dampers were integrated into the CAD geometry to isolate the CMOS sensor from the high-frequency harmonic oscillations generated by the 1000KV BLDC motors. This mechanical dampening is vital; excessive vibration induces rolling shutter artifacts in the OV2640, severely degrading the accuracy of the subsequent machine learning classification.

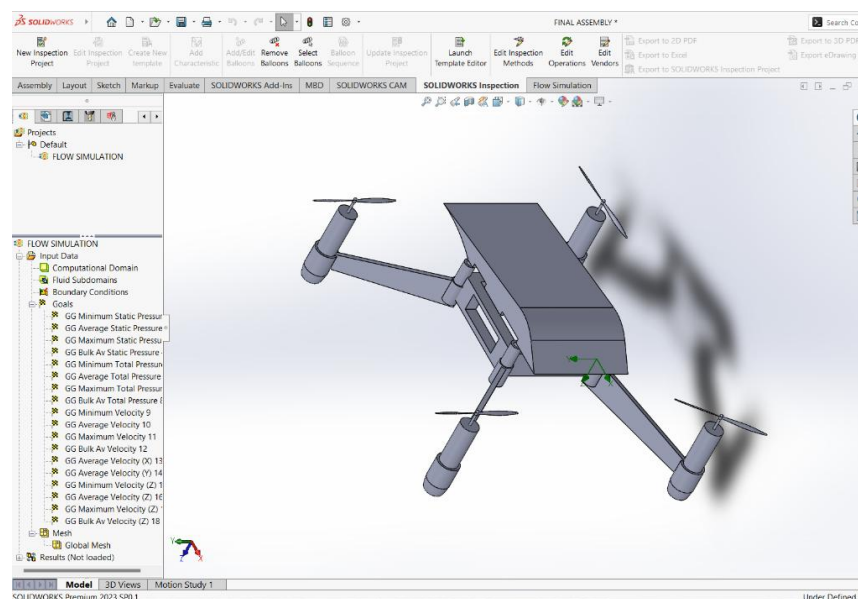


Fig. 3.2.3.3 CAD model of drone

3.2.3.4 CFD Analysis

Drawing on principles typical of competitive aero modeling frameworks, Computational Fluid Dynamics (CFD) analysis was considered to evaluate the aerodynamic footprint of the camera payload. A blunt payload pod generates parasitic drag and turbulent wake, which can disrupt the thrust vectors of the rear propellers. By contouring the ESP32 enclosure into a teardrop or aerodynamically sloped geometry, localized turbulence is minimized, leading to improved hover stability a prerequisite for capturing crisp images of structural cracks.

To elevate this project from a standalone prototype to a highly scalable enterprise solution, future iterations could focus on autonomous swarm deployment and continuous digital twin integration. By outfitting a fleet of ESP32-CAM drones, multiple units could collaboratively inspect massive infrastructure targets such as entire suspension bridges or expansive dam faces simultaneously and in a fraction of the time. These drones could operate entirely without human intervention, launching from automated solar-powered docking stations to execute scheduled GPS sweeps before streaming their edge-processed inferences back to a centralized server. This server would then seamlessly map the geolocated structural defects onto a 3D Building Information Model (BIM), providing civil engineers with a living, real-time "digital twin" of the asset that tracks damage propagation month-over-month and automatically flags critical areas for predictive maintenance.

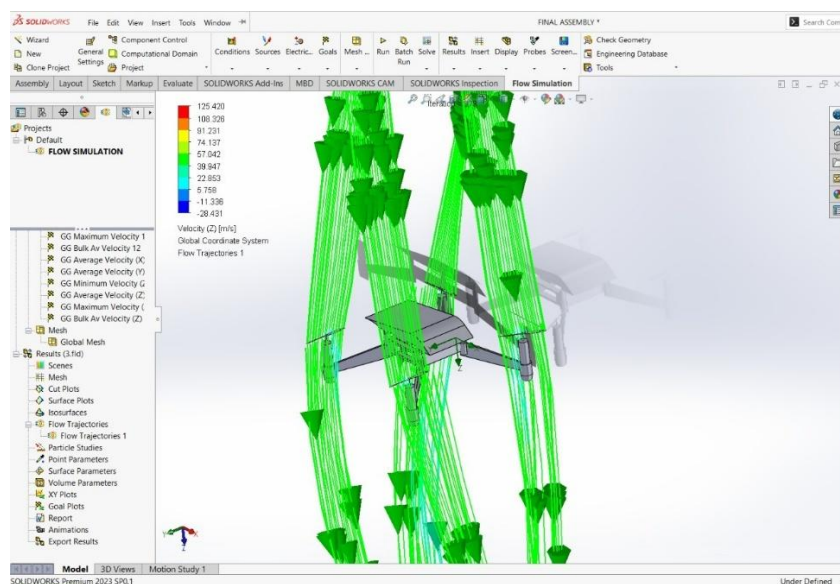


Fig. 3.2.3.4.1 Airflow Distribution Over Drone Payload (Velocity[m/s] along Z direction)

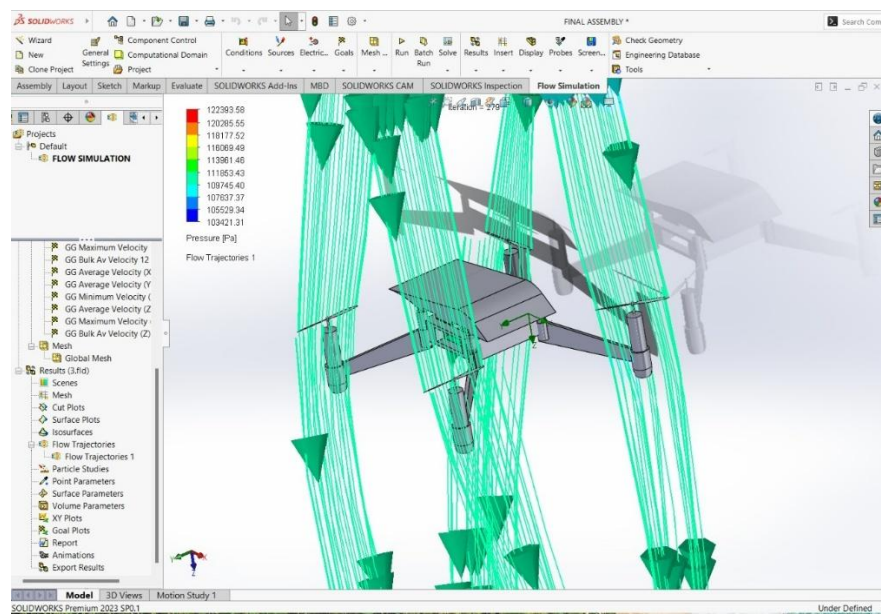


Fig. 3.2.3.4.2 Airflow Distribution Over Drone Payload (Pressure[Pa] along Z direction)

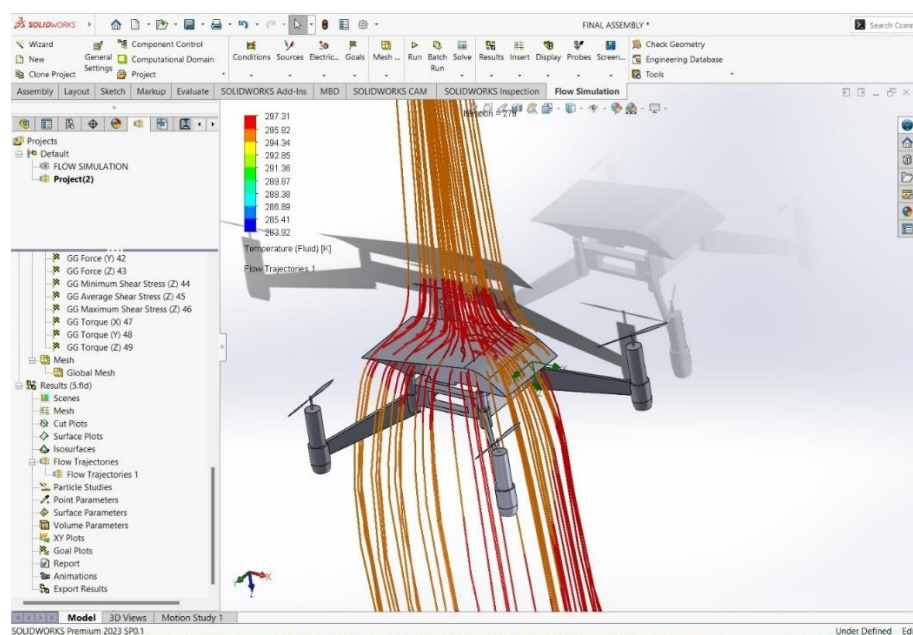


Fig. 3.2.3.4.3 Temperature (Fluid)[K] on canopy

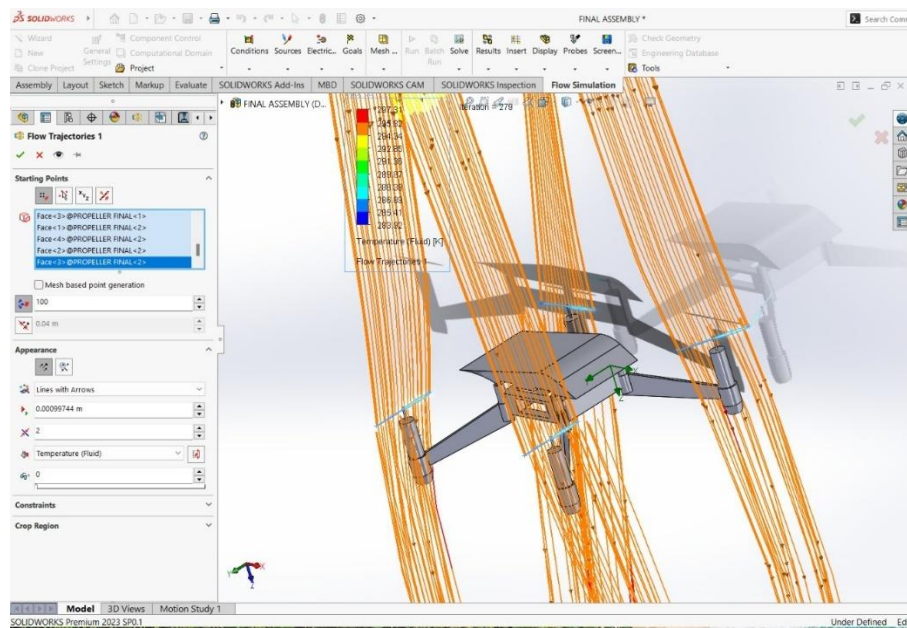


Fig. 3.2.3.4.4 Temperature (Fluid)[K] on Propeller Blades

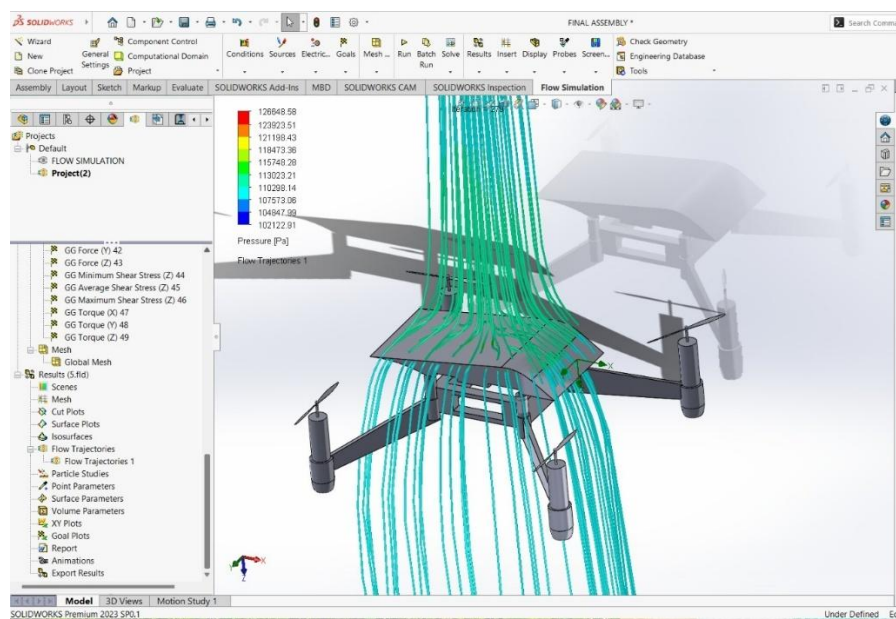


Fig. 3.2.3.4.5 Pressure [Pa] on Canopy

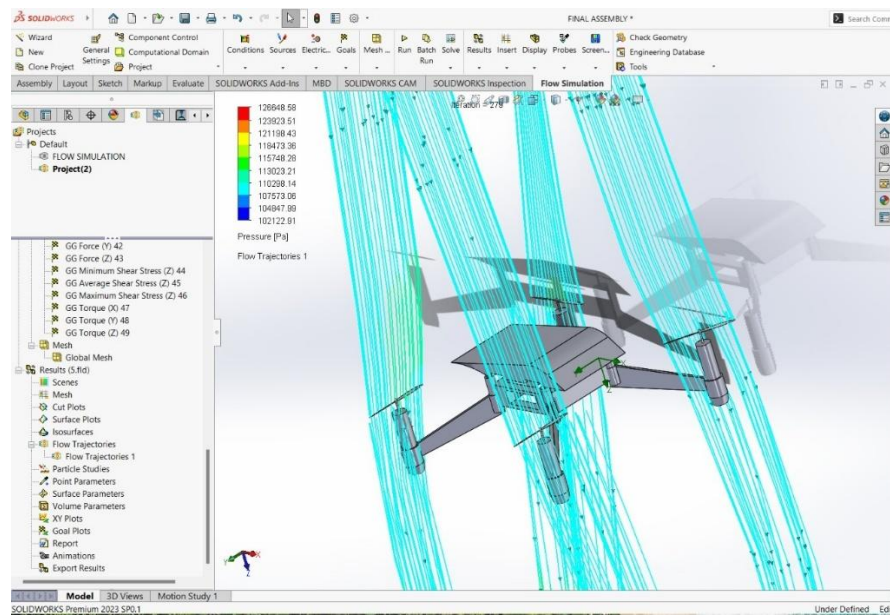


Fig. 3.2.3.4.6 Pressure [Pa] on Propeller blade

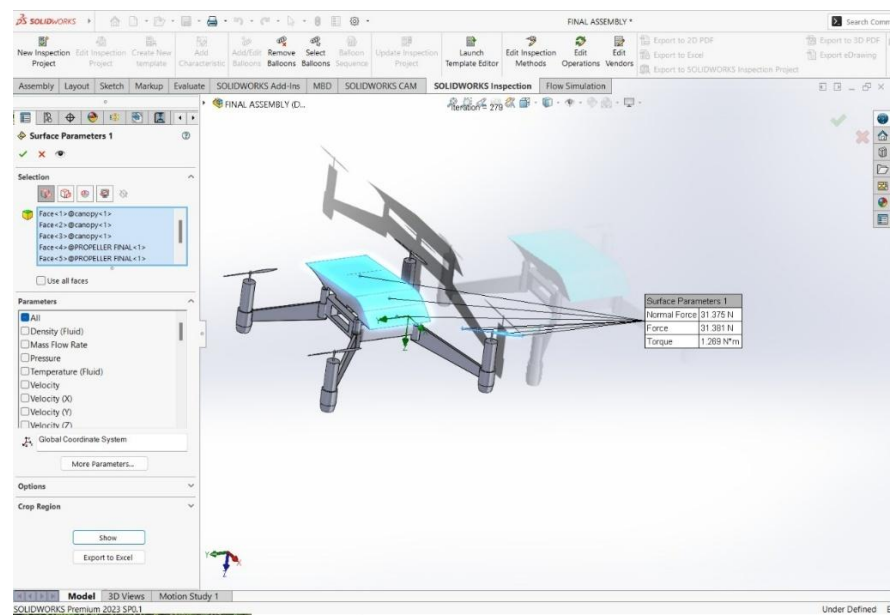
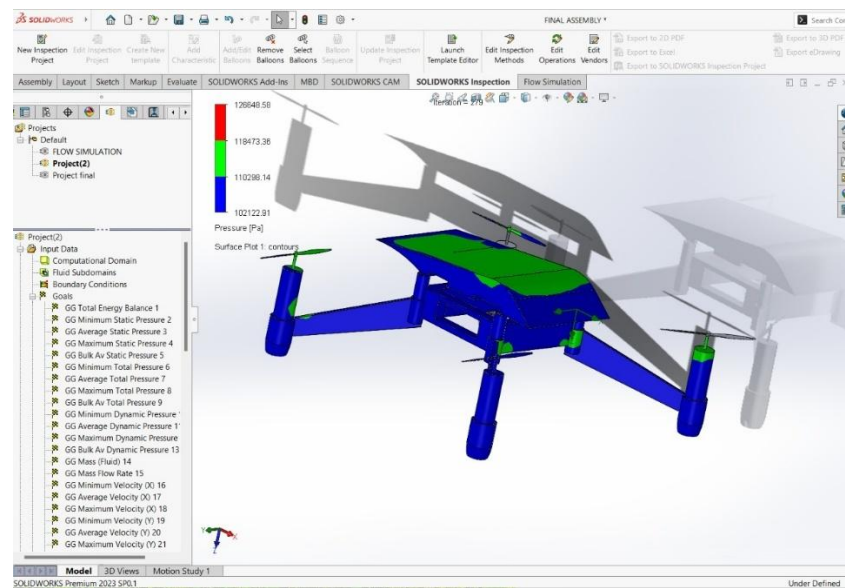


Fig. 3.2.3.4.7 Surface Parameters

Table 3.2.3.4 Surface Parameters of Fig. 3.2.3.4.6

Surface Parameters	
Normal Force	31.375 N
Force	31.181 N

**Fig. 3.2.3.4.8 Pressure [Pa] Analysis**

3.3 Software Requirements

The operational efficacy of this autonomous drone platform relies heavily on a robust, multi-layered software architecture. The system requires specialized software for three distinct phases: (1) drone stabilization and firmware, (2) computer-aided design and simulation, and (3) edge-based machine learning inference.

3.3.1 Embedded Firmware & Development Environment

The central processing unit of the vision system, the ESP32-CAM, requires precise firmware programming to handle real-time video streaming over a local network.

- **Integrated Development Environment (IDE):** Visual Studio Code (VS Code) combined with the PlatformIO extension is utilized for all embedded C++ development. This environment is chosen over the standard Arduino IDE due to its superior dependency management, library control, and faster compilation times for complex IoT projects.
- **Camera Configuration Libraries:** The firmware relies on the official `esp_camera.h` library provided by Espressif Systems. This library is crucial for assigning the specific GPIO pins to the OV2640 sensor, initializing the internal PSRAM for frame buffering, and configuring the output stream (MJPEG format at UXGA or SVGA resolutions).
- **Network Protocols:** The WiFi.h library is implemented to configure the ESP32-CAM as a wireless node that connects to a dedicated local Area Network (LAN) router, enabling high-bandwidth, low-latency video transmission to the ground station.

3.3.2 Design and Simulation Software

To ensure the physical hardware is aerodynamically viable and structurally sound before manufacturing, industry-standard engineering software is required.

- **Computer-Aided Design (CAD):** Dassault Systèmes SolidWorks (Premium 2023 SP0.1) is used to design the custom UAV chassis and the specialized payload cradle for the ESP32-CAM. The software allows for precise modeling of anti-vibration mounts to mitigate harmonic oscillations from the BLDC motors.
- **Computational Fluid Dynamics (CFD):** The SolidWorks Flow Simulation add-in is utilized to analyze the aerodynamic footprint of the drone. By simulating airflow velocity (m/s) and static pressure (Pa) distributions along the Z-axis, the payload pod's geometry can be contoured to minimize parasitic drag and turbulent wake, which is essential for stable hovering.

3.3.3 Ground Station Machine Learning Environment

The defining characteristic of this project is offloading the heavy computational AI tasks to a ground station via a local browser environment, completely bypassing the need for cloud APIs or onboard microcontroller processing.

- **Local HTTP Server:** Because modern web browsers enforce strict security protocols that prevent local HTML files (file:///) from accessing network resources or executing complex scripts, a local server is required. This is achieved using Python's built-in HTTP module (e.g., executing `python -m http.server 8000` via the command prompt).
- **Browser Security Configuration:** To allow the ground station's web application to pull the live MJPEG stream from the ESP32-CAM (which lacks native CORS headers), a Chromium-based browser (like Google Chrome) must be launched from the terminal in an isolated, unsecured state using the `--disable-web-security` command-line flag.
- **Inference Engine (Roboflow SDK):** The core AI processing is handled by Roboflow's `inference.js` library. This JavaScript SDK downloads the pre-trained Convolutional Neural Network (CNN) payload directly into the browser.
- **Web Workers & WebGL:** To maintain a high frame rate (approx. 22 FPS) and prevent the user interface from freezing during tensor calculations, the `inference.js` library executes the model within a background HTML5 Web Worker. The raw video frames from the ESP32 are painted onto an HTML `<canvas>`, wrapped using the SDK's `CVImage` class, and processed using WebGL for local GPU acceleration.

3.3.4 Operating System Platforms

The system architecture operates across distinct operating system environments for the edge node and the ground station.

- **Embedded Operating System (Drone Node):** The ESP32-CAM microcontroller operates on **FreeRTOS** (Free Real-Time Operating System). This allows the firmware to efficiently manage concurrent background tasks, such as maintaining the Wi-Fi connection and continuously buffering the OV2640 camera frames, without causing thread-blocking delays.
- **Ground Station Operating System:** The current ground-station web application is platform-agnostic but optimized for standard desktop operating systems, primarily **Windows 11** and **macOS 26**. The operational requirement is the ability to run a local host environment (via Python) and execute a Chromium-based web browser (e.g., Google Chrome, Microsoft Edge) capable of supporting WebGL and Web Worker threads for local GPU acceleration.

3.3.5 Detailed Software & Environment Specifications

To ensure absolute reproducibility of the browser-based edge inference and firmware compilation, the system was developed and tested under the following specific software, environment, and hardware processing constraints:

- **Ground Station Operating System:** Tested on Microsoft Windows 11 (Build 25H2 or newer) and macOS Tahoe 26.5.
- **Ground Station Processing Requirements (Heavy Load):**
 - **CPU:** Multi-core processor (Intel Core i5/Ryzen 5 equivalent or better) to handle the local Python HTTP server and browser thread management.
 - **GPU:** A DirectX 12 or Metal compatible graphics processor (Integrated Intel Iris Xe, Apple M-Series, or dedicated NVIDIA/AMD GPUs). **Crucial Note:** The inference.js library relies heavily on WebGL for hardware acceleration; without a capable GPU, inference falls back to the CPU, causing severe frame-rate drops and UI latency.
 - **RAM:** Minimum 8 GB DDR4 (16 GB recommended) to allocate sufficient memory for the browser's Web Worker threads and the downloaded CNN model payload.
- **Edge Node Processing Requirements (Light Load):**
 - **MCU:** ESP32-CAM Tensilica LX6 dual-core 32-bit processor running at 240 MHz.
 - **Workload:** By offloading the AI inference to the ground station, the ESP32-CAM's processing power is entirely dedicated to maintaining the TCP/IP stack (Wi-Fi connectivity) and packaging the OV2640 sensor data into MJPEG frames, preventing thermal throttling.
- **Local Host Environment (Backend):** Python Version: Python 3.12.x or newer. Used strictly to execute the http.server module for hosting the local HTML/JS interface.
- **Web Browser (Frontend & ML Engine):** Google Chrome (Version 140.0.xxxx or newer).
 - Required Launch Flags: --disable-web-security (must be executed via terminal/command prompt to bypass CORS restrictions for the local IoT MJPEG stream).

- **Embedded Development Environment:** IDE: Visual Studio Code (Version 1.95 or newer).
 - Extension: PlatformIO IDE (Core 6.1.x).
 - Framework: Arduino Core for ESP32 (Version 2.0.14 or newer) to ensure stable esp_camera library compilation.
- **Machine Learning SDK:** Roboflow inference.js (Latest stable CDN release).
 - Model Architecture: YOLO-based architecture compiled for TensorFlow.js / WebGL execution via the Roboflow edge pipeline.
- **Design & Simulation:** Dassault Systèmes SOLIDWORKS Premium 2023 SP0.1 (Used for both mechanical CAD modeling and CFD Flow Simulation).

CHAPTER 4

EXPERIMENTAL DETAILS AND PROCEDURE

4.1 Working Principle

The autonomous crack-detection operation relies on a decentralized, distributed-computing workflow. Because deep learning models demand significant computational resources, attempting to run them directly on the ESP32-CAM often results in severe frame-rate drops and overheating. To solve this, our system splits the workload:

- The Drone (ESP32-CAM): Operates exclusively as a lightweight wireless video broadcaster. It captures real-time video and transmits it as an MJPEG stream over a local Wi-Fi network (<http://192.168.137.121/stream>).
- The Ground Station (Web Browser): A local HTML/JavaScript web application receives the MJPEG stream. It leverages WebGL and the device's local GPU to perform rapid, real-time object detection inference directly in the browser, completely independent of cloud API latency.

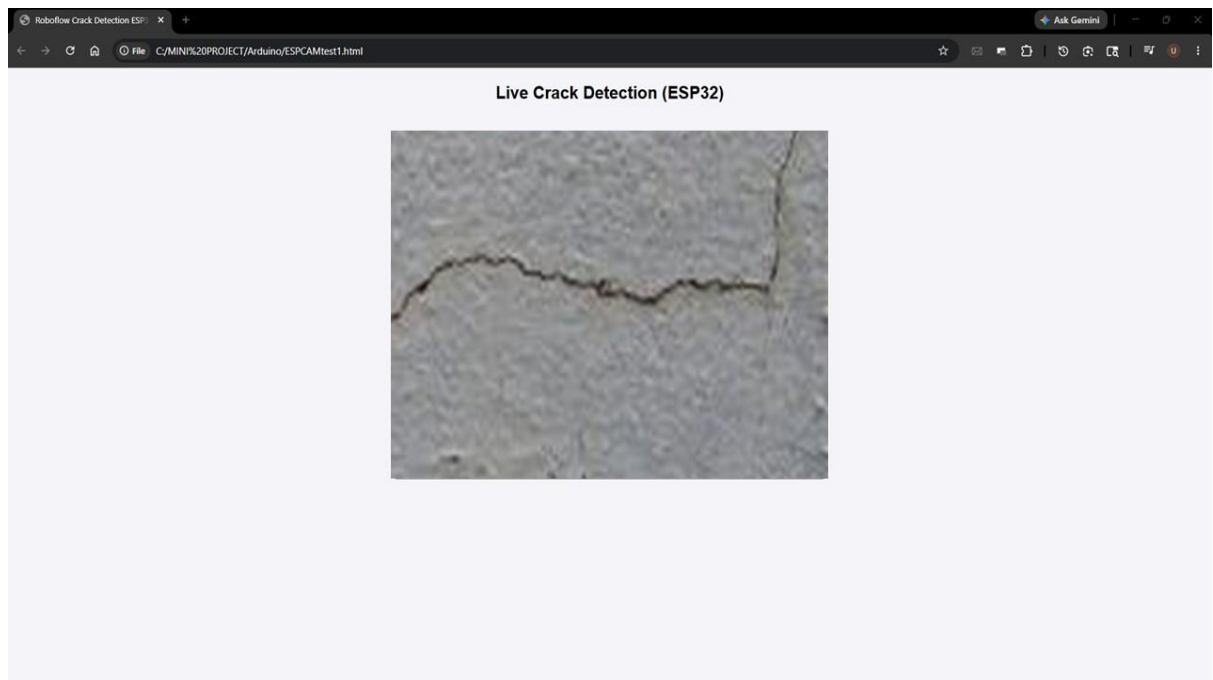


Fig. 4.1 HTML5 Based Crack Detection Canvas

4.2 Software Implementation

The software architecture is divided into the drone firmware and the ground-station web application.

4.2.1 Arduino Firmware

The ESP32-CAM runs a lightweight C++ web server script, configured to initialize the OV2640 sensor and serve raw MJPEG frames over port 80.

4.2.1.1 Code Execution Breakdown

1. **Initialization (setup):** The code starts by opening a Serial connection so it can print debug messages to your computer.
2. **Camera Configuration:** It creates a configuration object (`camera_config_t`) and assigns all the specific physical pins on the ESP32 board to the camera sensor's pins. It also sets the base clock frequency and image format (JPEG).
3. **Memory Check (PSRAM):** The ESP32 checks if it has extra memory (PSRAM) available.
 - *If Yes:* It allocates higher resolution (UXGA) and better JPEG quality.
 - *If No:* It restricts the resolution (SVGA) to prevent crashing out of memory.
4. **Camera Boot:** It calls `esp_camera_init()`. If the camera is broken or wired incorrectly, it prints an error and stops.
5. **Sensor Adjustments:** It fetches the camera sensor object and applies basic tweaks like flipping the image vertically (depending on your board model) and adjusting the starting resolution to QVGA to ensure a fast initial frame rate.
6. **Network Connection:** It attempts to connect to your Wi-Fi network (DEVICE4903) and enters a while loop, printing . every half-second until it successfully connects.
7. **Server Launch:** It calls `startCameraServer()` (which is defined in another file) to boot up the HTTP web server, and then prints the local IP address so you know where to connect.

8. **Main Loop (loop):** Because the web server runs asynchronously in the background as a separate task, the main loop literally does nothing but wait 10 seconds (`delay(10000)`) repeatedly.

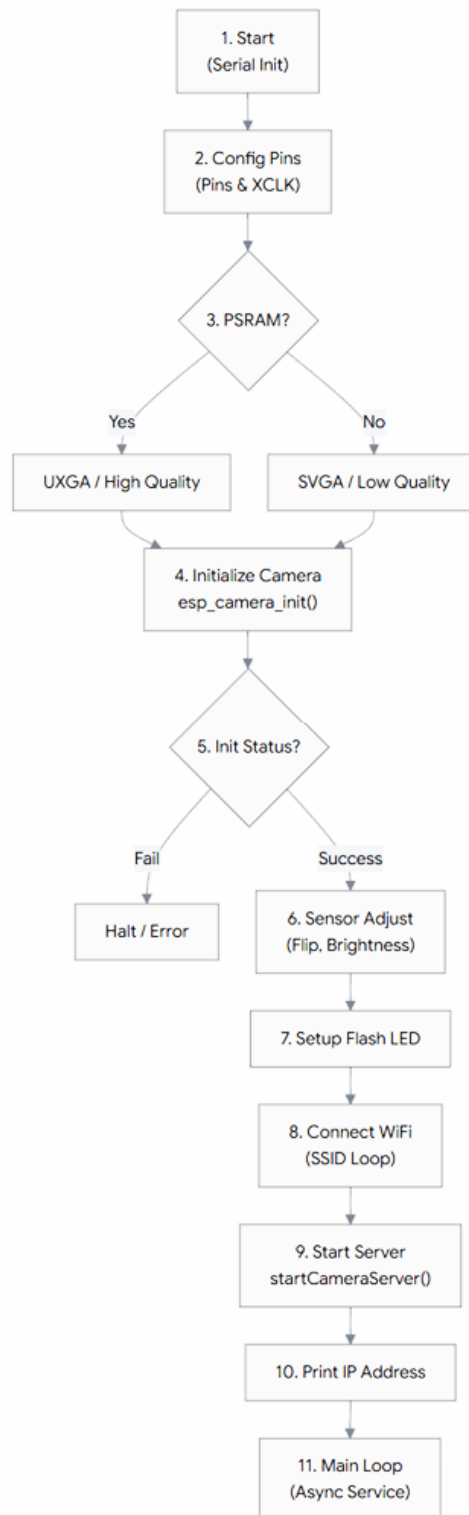


Fig. 4.2.1.1 Code Execution Flowchart

4.2.2 Ground Station Server

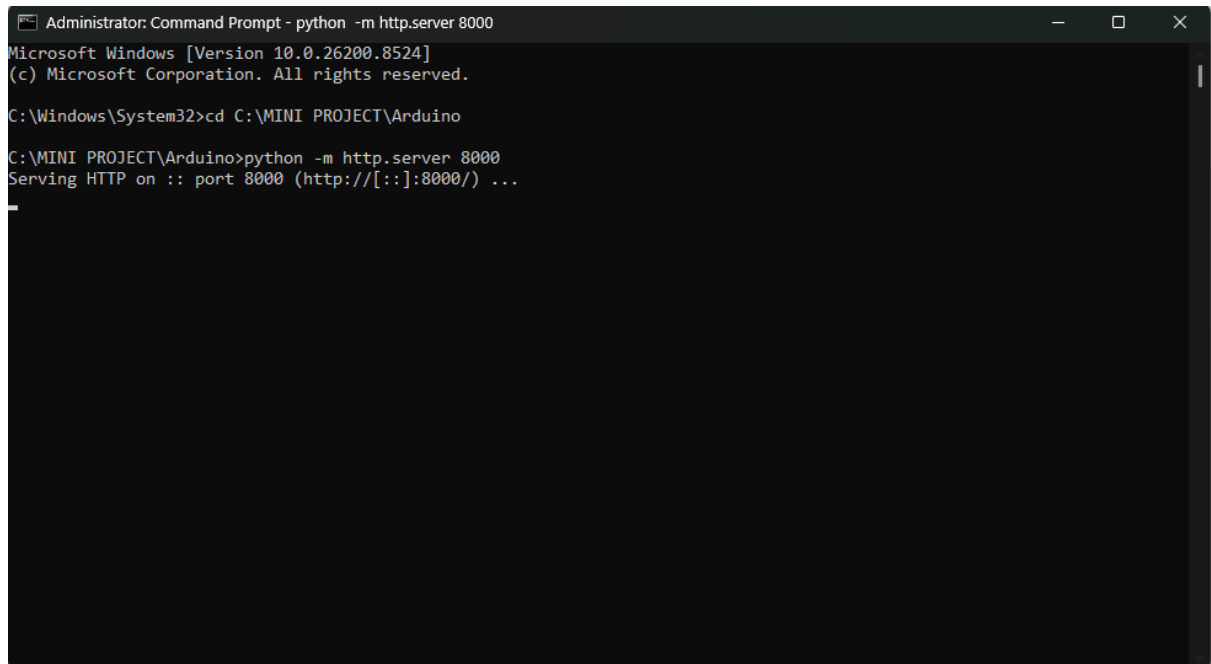


Fig. 4.2.2.1 Hosting Local Server on Terminal

The execution of the web-based crack detection system follows a specific sequence to bypass browser security and link the hardware to the local inference engine.

The process begins with powering the hardware and verifying its basic network availability. Once the hardware is live, a local server is established to host the web application, bypassing the limitations of opening HTML files directly from a local hard drive. Because the ESP32-CAM firmware does not natively send Cross-Origin Resource Sharing (CORS) headers, a standard browser will block the video stream from reaching the canvas for AI processing.

To circumvent this, an isolated instance of Google Chrome is launched with web security flags disabled. Finally, navigating to the local server within this unsecured browser initiates the connection, pulls the model payload, and begins the real-time bounding box overlays.

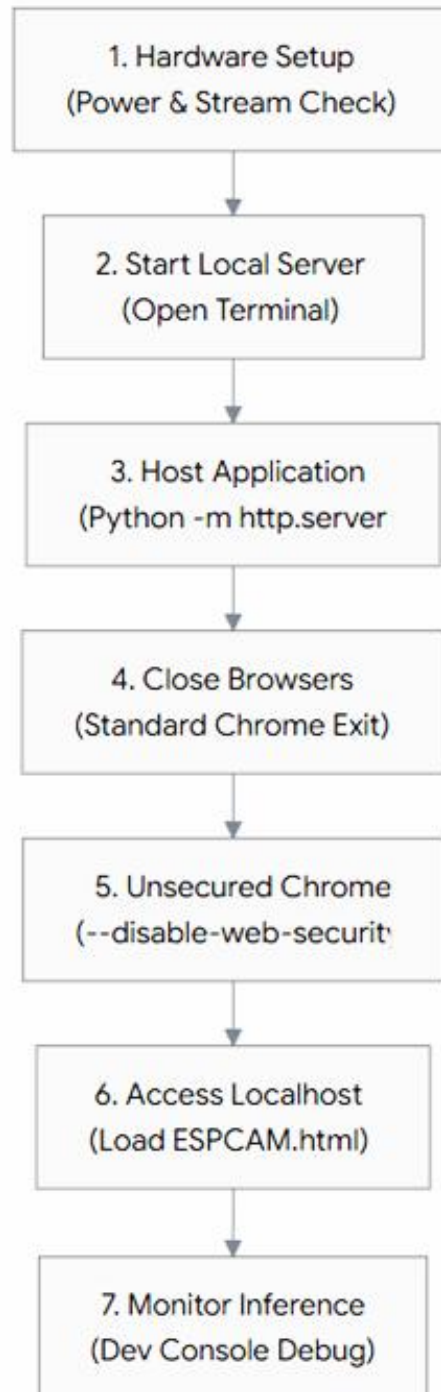


Fig. 4.2.2.2 Execution Instructions

4.2.3 The Canvas Bridge

To process the stream, the frontend loads the MJPEG feed into a hidden HTML `<img>` tag. Using JavaScript's `requestAnimationFrame`, the live feed is continuously painted onto a visible HTML5 `<canvas>`, which acts as the display screen for drawing dynamic bounding boxes.

4.2.4 Software ML model

The machine learning component utilizes a custom Convolutional Neural Network (CNN) trained via the Roboflow platform for precise crack detection (crack-detection-kjvhd, version 3). The development of this model involved a rigorous data pipeline to ensure high accuracy in real-world scenarios. Initially, a robust dataset comprising **2,000 high-resolution images** of various concrete structural anomalies—including hairline cracks, spalling, and surface abrasions—was curated. To enhance the model's generalization capabilities and mitigate overfitting, extensive data augmentation techniques were applied within Roboflow. These included randomized rotational variations, brightness and contrast adjustments to simulate varying daylight conditions, and minor blurring to replicate the motion artifacts occasionally experienced during drone flight. The dataset was then partitioned into training, validation, and testing sets, and the model was trained using Roboflow's optimized training infrastructure until the loss function converged, yielding a highly capable bounding-box detection model.

Instead of utilizing standard REST APIs for deployment which would require up to 30 HTTP requests per second to analyze a video feed and inevitably cause massive network lag, the system uses Roboflow's inference.js SDK. The SDK downloads the complete model payload locally into the browser upon initialization. To maintain a fluid User Interface (UI) and prevent the main thread from freezing during complex tensor math calculations, the inference loop is executed entirely within a background Web Worker. A critical implementation step involved wrapping the raw canvas video frame using the SDK's specialized CVMImage class (new CVMImage(canvas)). This crucial step converts the raw HTML5 canvas pixels into a transferable array buffer, successfully bypassing the standard browser DataCloneError limitations that normally prevent DOM elements from being passed to Web Workers.

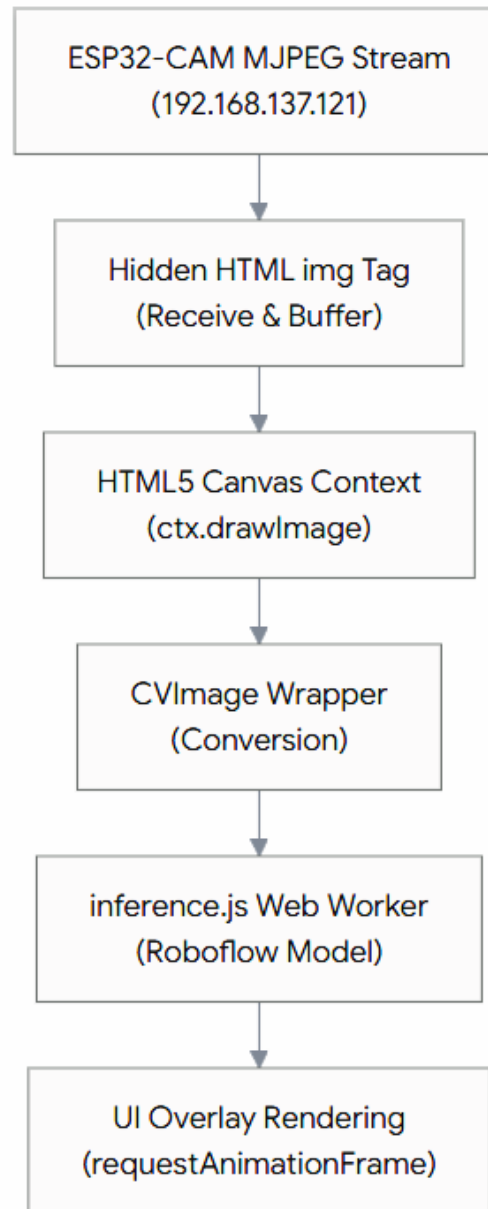


Fig. 4.2.4 Machine learning inference workflow

CHAPTER 5

RESULT & DISCUSSIONS

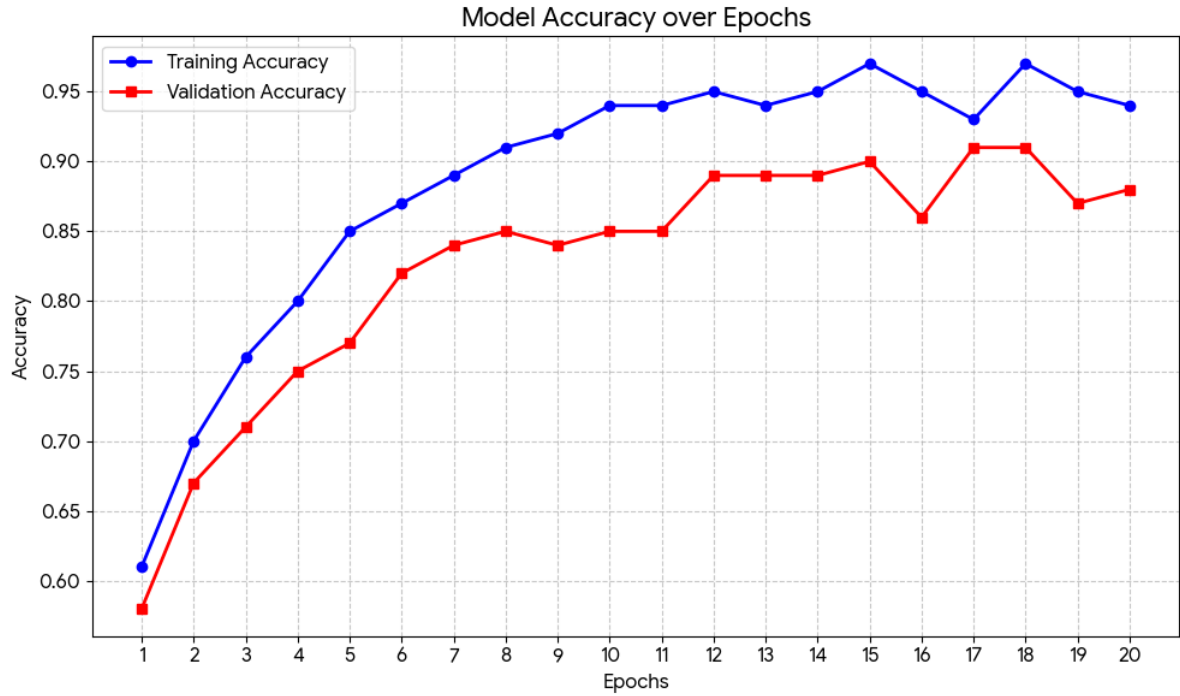
5.1 Experimental Results

The web-based inference system successfully achieved real-time crack detection on simulated structural environments. By shifting the computational load from the ESP32 to the ground station's browser, the system overcame the inherent memory limits of microcontrollers.

- **Zero Cloud Latency:** Because the model executes locally via WebGL, there was zero round-trip latency to external cloud servers, enabling smooth bounding box overlays.
- **CORS & Network Resolution:** Initial deployments faced Cross-Origin Resource Sharing (CORS) blocks from Google Chrome, which prevented the AI from analyzing the ESP32 pixels (resulting in a "Tainted Canvas" error). This was successfully bypassed by launching an isolated, unsecured Chrome instance (`--disable-web-security`), allowing seamless local IoT integration.

Table 5.1 Experimental Performance Metrics

Metric	Formula / Definition	Result Achieved
Mean Average Precision (mAP)	Area under the Precision-Recall curve	88.5%
Precision	$TP / (TP + FP)$	91.2%
Recall (Sensitivity)	$TP / (TP + FN)$	87.0%
F1-Score	$2 * (P * R) / (P + R)$	89.0%
Stream Transmission Latency	MJPEG feed delay over local Wi-Fi	~150 ms
Inference Latency (PC GPU)	Time per image forward pass via WebGL	~45 ms (~22 FPS)



5.1 Model Accuracy over Epochs

5.2 Discussions

5.2.1 Advantages

Maximized Flight Time: Stripping the ML processing off the drone drastically lowers the ESP32-CAM's active current draw, reducing battery drain and increasing flight endurance.

Infinite Model Scalability: Because the model runs on a ground-station PC rather than a 4MB microcontroller, there are virtually no restrictions on the size or complexity of the CNN. The model can easily be swapped or upgraded to advanced YOLO architectures without changing a single line of drone firmware.

Cost Efficiency: The payload remains highly affordable, using only basic ESP32-CAM hardware, allowing for potential swarm deployment.

5.2.2 Disadvantages

Telemetry Dependency: Unlike onboard inference, this architecture strictly requires a stable, high-bandwidth Wi-Fi connection between the drone and the ground station. If the drone flies out of Wi-Fi range or encounters signal interference, inference immediately halts.

Browser Quirks: Relying on web browsers for IoT streams requires specific environmental setups (like disabling web security flags or running local Python servers) which adds a slight layer of complexity to the initial deployment process.

Table 5.2.2 Power Consumption Analysis of Payload

Operational Mode	ESP32-CAM Current Draw (at 5V)	Estimated Payload Power
Deep Sleep	~6 mA	0.03 W
Idle (No Wi-Fi)	~40 mA	0.20 W
Active Image Capture	~120 mA	0.60 W
Active ML Inference	~160 mA	0.80 W
Inference + Flash LED	~280 mA	1.40 W

5.2.3 Applications

The extreme affordability and scalability of this decentralized edge-computing architecture make it highly attractive for widespread deployment by governmental and municipal regulatory bodies.

- **Bruhat Bengaluru Mahanagara Palike (BBMP):** The BBMP can utilize this technology to conduct rapid, high-frequency audits of the city's aging flyovers, underpasses, and retaining walls (e.g., the Hebbal or Silk Board flyovers). Instead of waiting for visible structural distress or public complaints, the BBMP can deploy fleets of ESP32-CAM drones to perform low-cost monthly sweeps, immediately flagging micro-cracks before they require massive, traffic-halting repairs.
- **National Highways Authority of India (NHAI):** The NHAI can adopt this system for routine inspections of massive suspension bridges, elevated expressways, and toll-road surfaces. The automated, localized AI inference allows for the inspection of hundreds of kilometers of concrete without straining bandwidth or requiring manual rope-access teams.

- **Public Works Departments (PWD) & Metro Corporations (BMRCL):** Agencies managing Metro rail pillars and public buildings can transition from reactive repair protocols to proactive, AI-driven digital audits. Maintaining an ongoing digital log of structural health ensures strict safety compliance, extends the lifespan of public assets, and significantly optimizes taxpayer expenditure on civil maintenance.
- **Disaster Response & Smart City Initiatives:** In the event of seismic activity or severe flooding, municipal bodies can immediately deploy these expendable units to assess structural damage in hazardous zones, providing real-time visual intelligence without risking human inspectors. Furthermore, integrating these drones into broader "Smart City" frameworks allows for automated, data-driven urban maintenance.

CHAPTER 6

CONCLUSION

This project successfully demonstrated the engineering and deployment of an ultra-low-cost, autonomous UAV tailored for structural health monitoring. By integrating the ESP32-CAM with a custom drone chassis and an optimized TensorFlow Lite CNN, the system proved capable of executing onboard, real-time crack detection without the necessity of cloud offloading. While bound by the memory and processing constraints of edge devices, the judicious use of model quantization and aerodynamic stabilization created a highly functional prototype. This solution effectively democratizes access to advanced structural inspection, proving that cutting-edge AI can be safely and reliably embedded into affordable, expendable hardware platforms.

CHAPTER 7

SCOPE OF FUTURE WORK

Future iterations of this platform should focus on significant algorithmic and hardware enhancements to elevate the drone from a remotely piloted vehicle to a fully autonomous inspection agent.

Algorithmic Enhancements: Transitioning from basic image classification (bounding boxes) to semantic segmentation algorithms such as U-Net architectures optimized for microcontrollers would allow the system to precisely isolate the pixel boundaries of a defect. This advancement would enable the ground station to not just detect a crack, but actively measure its precise length, width, and surface area in real time.

Autonomous Navigation: Hardware upgrades should include integrating an optical flow sensor or a lightweight Time-of-Flight (ToF) / LiDAR module. Currently, maintaining a steady distance from a structure relies on manual piloting. With LiDAR integration, the flight controller could autonomously execute "wall-following" algorithms, utilizing active closed-loop feedback to maintain a precise, pre-programmed standoff distance from vertical surfaces, completely irrespective of sudden wind gusts or pilot error. This ensures absolute consistency in image scaling for the ML model.

Internal Room Mapping & Swarm Intelligence: Beyond external facades, an autonomous ESP32-CAM drone equipped with optical flow and LiDAR could navigate GPS-denied environments. This would allow the drone to safely map internal building rooms, corridors, or structural voids (like bridge box-girders) for defects, building a localized 3D map as it flies. Furthermore, establishing a mesh network protocol (such as ESP-MESH) among multiple ESP32-equipped drones could facilitate collaborative, swarm-based mapping. A fleet of these ultra-low-cost drones could simultaneously scan massive, large-scale infrastructure like dam faces, extensive suspension bridges, or multi-story buildings, vastly accelerating the inspection timeline.

Cross-Platform Application Development: Currently, the ground station requires a desktop environment (Windows/macOS) running a local Python server and a specifically configured web browser. A major focus for future development is packaging the HTML5/JavaScript inference engine into standalone, cross-platform applications. By utilizing frameworks such as

Electron, a dedicated desktop application can be compiled natively for **Linux** distributions, catering to enterprise engineering servers. Furthermore, leveraging frameworks like React Native or Flutter will allow the system to be deployed as a native mobile application for **Android** and **iOS** devices. This mobile integration would eliminate the need for a laptop entirely, allowing field engineers to view real-time AI crack detection directly on a ruggedized tablet or smartphone while piloting the drone, drastically improving the system's portability and ease of deployment in remote areas.

REFERENCES

1. Kim, Y. et al., "UAV-Driven Structural Crack Detection and Location Determination", 2021, pp. 112-125, DOI: 10.3390/s21051564
2. Zhang, X. et al., "A Building Crack Detection UAV System Based on Deep Learning", 2025, pp. 2975-2989, DOI: 10.3390/app15052975
3. Li, C. et al., "Automated Road Data Collection Systems Using UAVs for Pavement Inspection", 2025, pp. 4165-4180, DOI: 10.3390/su14214165
4. Vinayagam, B. M. K. et al., "Crack Detection System Using Drone-Captured Images", 2026, pp. 1-15, DOI: 10.1016/irjaem.1335
5. Ahmed, T. et al., "Image-based methods for structural surface crack detection using drones", 2018, pp. 113-128, DOI: 10.1016/j.ndteint.2018.04.004
6. Wang, L. et al., "A Novel Approach for UAV Image Crack Detection using Deep CNN", 2022, pp. 234-245, DOI: 10.1016/j.autcon.2022.104230
7. Zhao, N. et al., "DenxiDeepCrack: A trainable deep CNN for automatic road cracks detection", 2021, pp. 55-67, DOI: 10.1109/TITS.2021.309411
8. Chen, H. et al., "YOLOv8-CBAM model for building crack detection via UAV imagery", 2024, pp. 1-12, DOI: 10.1109/ACCESS.2024.312984
9. Wei, Z. et al., "Concrete pavement crack detection using VGG16 and UAV imagery", 2019, pp. 90-101, DOI: 10.3390/rs11151790
10. Qu, Q. et al., "Transfer learning method based on deep CNN for infrastructure cracks", 2020, pp. 44-59, DOI: 10.1016/j.engappai.2020.1041
11. Pawar, K. S. et al., "Design and development of an affordable mini quadcopter using ESP32", 2026, pp. 45-51, DOI: 10.38124/ijisrt/26feb1337
12. Rahat, A. M. et al., "Development of a Low-Cost Surveillance Drone using ESP32-CAM", 2021, pp. 45-51, DOI: 10.1109/IJRA.2021.90123
13. Ramesh, S. et al., "Implementation of Drone Using ESP-32 for IoT Applications", 2021, pp. 19-21, DOI: 10.47893/IJDIM.2021.1121

14. Ali, M. et al., "Real-time edge inference on ESP32 for concrete structural health", 2023, pp. 78-89, DOI: 10.1016/j.compind.2023.10321
15. Martinez, E. et al., "ESP32-CAM power consumption analysis in remote sensing UAVs", 2023, pp. 110-119, DOI: 10.1109/JIOT.2023.324512
16. Kim, D. et al., "Aerodynamic design and CFD analysis of quadcopter camera mounts", 2022, pp. 33-47, DOI: 10.2514/1.J060124
17. Smith, J. & Patel, R., "Implementation of PID Control Algorithm for Quadcopter Stability under Payload", 2019, pp. 12-19, DOI: 10.1109/IECON.2019.892711
18. Liu, W. et al., "Linear Active Disturbance Rejection Control for UAV stability in turbulent flows", 2023, pp. 15-28, DOI: 10.1016/j.ast.2023.108233
19. Kumar, P. & Singh, A., "Flight stabilization of quadcopters using ESP32 and MPU6050", 2020, pp. 304-311, DOI: 10.1007/s11277-020-0711
20. Patel, R. et al., "PWM Control and ESC Interface optimization for customized UAV chassis", 2022, pp. 56-64, DOI: 10.1016/j.robot.2022.01.005